Slide 1

CS 170
Java Programming 1
Introducing Arrays
Duration: 00:01:55
Advance mode: Auto

Notes:

Hi Folks. Welcome to the CS 170, Java Programming 1 lecture, Introducing Arrays.

The Atacama Large Millimeter Array, (ALMA) is an array of 64 radio antennas that work together as one telescope to study millimeter-and sub-millimeter-wavelength light from space. ALMA is being built in Chile by an international partnership which includes the National Research Council of Canada and the U.S. National Radio Astronomy Observatory. It is the largest ground-based astronomy endeavor ever undertaken.

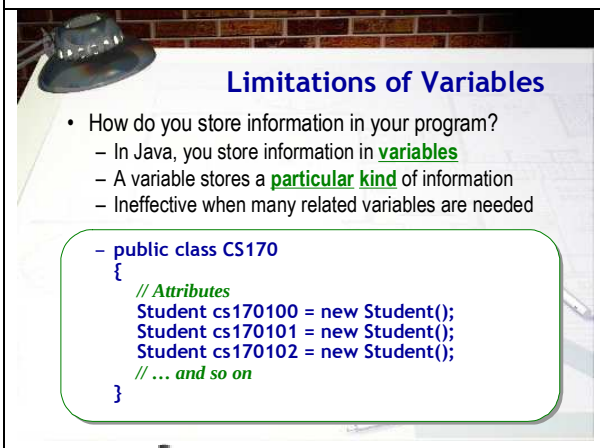
In Java, an array is similar. Arrays are the built-in Java type for holding collections of variables. Arrays are useful whenever you need to apply identical processing to all of the members of a group of related variables. You could use an array, for instance, to calculate the grades for an entire class of students, rather than performing the operation one-by-one.

In the online lectures this week you'll learn:

- what array variables are, and why they're useful
- how to define and initialize array variables
- how to store and retrieve values using an array
- how to process arrays with methods and loops.

In this lecture I'll focus on creating and initializing arrays of primitive types. In the later lessons, you'll learn how to create and use collections of object types.

The Java library actually contains many different kinds collection classes, which are commonly known as data structures or abstract data types. Chapter 15 and 16 in your textbook, (which we won't cover in this class), have an introduction to the most commonly used classes. If you're a Computer Science major, you'll take CS 200, Data Structures, which uses similar classes from the C++ Standard Template Library (STL).



Slide 2

Notes:

Variables, you'll recall, are 'buckets' that hold a single value; you create variables to hold the data that your program processes and turns into information. When you create one of these "buckets", you have to say what kind of thing you're going to store in it; there are no "one size fits all" buckets, like there are in some languages.

Now, the only problem with variables comes when you want to store a lot of values; creating individual variables for each value can quickly become very tedious--especially if you have a lot of similar values that are somehow related. A program that keeps the records for a class of CS 170 Java students, for instance, would need to declare variables for

Limitations of Variables
Duration: 00:01:02
Advance mode: Auto

each student, something like this.

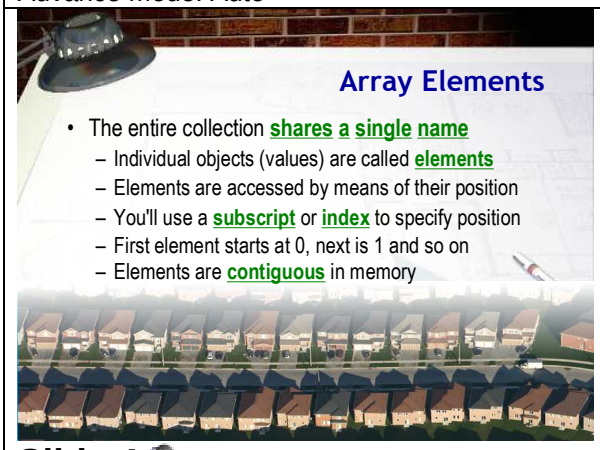
While you certainly could write code like this, imagine how long and unwieldy your program would become when it came time to add the scores for a quiz, for instance. You'd need to have a separate statement for each student variable. There has to be a better way, and there is.

Slide 3

What Are Arrays?
Duration: 00:00:32
Advance mode: Auto

Notes:

The solution that's built-in to the Java language is called the array. An array is a collection variable; an array can hold multiple values rather than just a single value. An array can store values of any type—either primitive values or object references—but each of the values in a single array must all be of the same type. You can't put eggs in a muffin-tin or muffins in an egg crate (so to speak.)



Slide 4

Array Elements
Duration: 00:02:17
Advance mode: Auto

Notes:

When you create an array, you give the entire collection a name, just like you would a regular variable. I think of it like the name of a neighborhood like Newport Heights or Turtle Ridge.

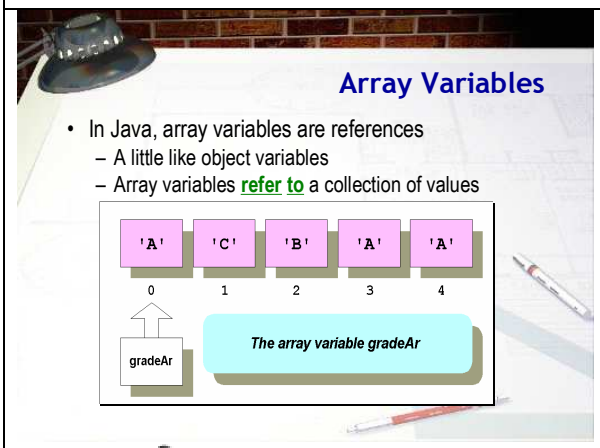
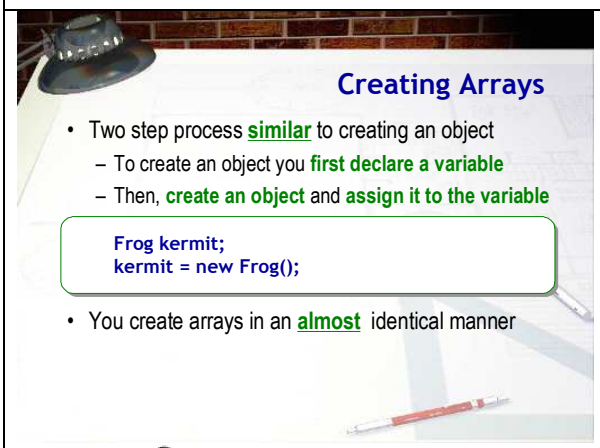
Just like you would indicate the houses in a neighborhood by their street number, you access the individual data values stored in the array—these are called the array elements—by specifying an index or subscript representing its position in the collection.

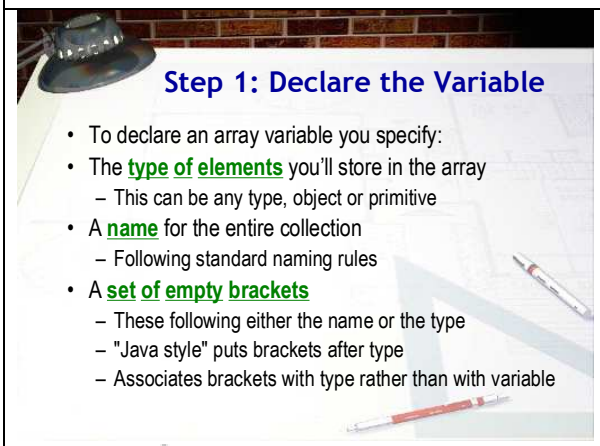
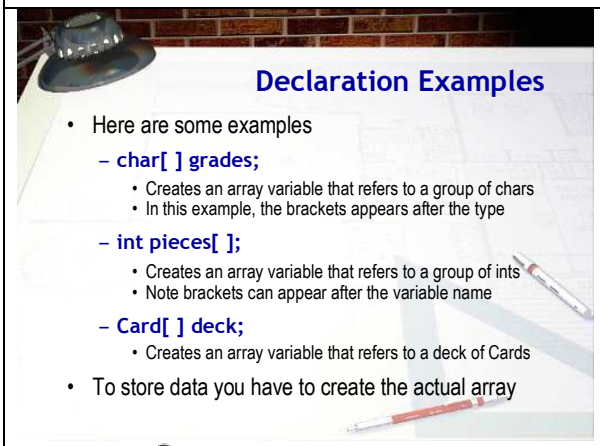
In Java, the first element of the array has the subscript 0, the next 1, and so on.

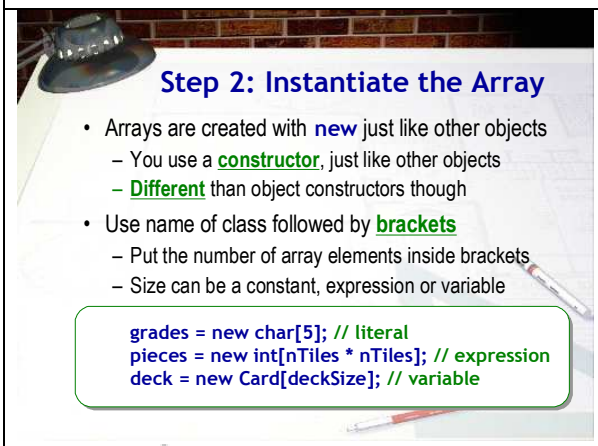
Why start at 0, you wonder? Good question. One of the perennial debates in programming language circles has to do with whether array elements should be numbered starting with zero or one. Java, like C and C++, uses zero-based arrays. While you might not like this, it may help a little if you understand why zero-based arrays are used.

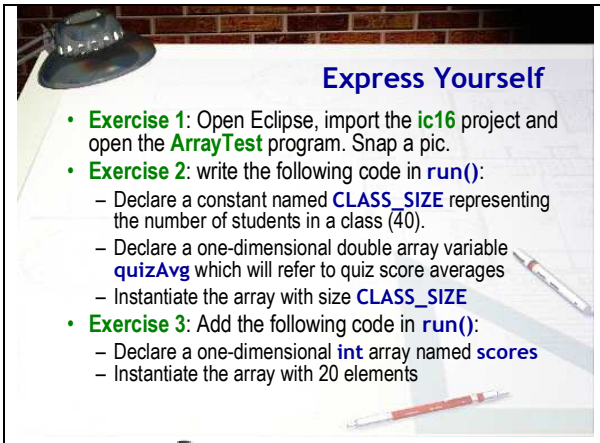
If you think of the array subscript as the number of the element—#1, # 2, etc., something like a house number—then zero based arrays make no sense at all. After all, what does it mean to be the "zeroth" element?

But, array subscripts are not the number of the element; instead, the subscript is an offset from the beginning of the array. When you tell the compiler to retrieve myArray[5], you

	are telling it to go to the array named myArray, then "walk past" five elements and bring you the element it finds there. If you tell it to access myArray[0], the compiler knows it doesn't have to skip any elements at all, and it brings you the first element. When you create an array, all of the elements are stored right next to each other in memory; we say that the elements are contiguous. Since each value must be the same type, each value uses exactly the same amount of memory, which allows Java to locate an individual value very quickly using simple arithmetic. If the first element is at address 100 in memory, and each element is 4 bytes long, the 1000th element must be at 100 + 4 * 1000.
 **Slide 5** **Array Variables** Duration: 00:01:01 Advance mode: Auto	**Notes:** In Java, arrays variables are references, just like object variables. This is quite a bit different than arrays in C, C++, Pascal or Fortran. It is similar to how arrays are represented in C#, however. An array variable does not actually contain the individual values in the array, it refers to them. Thus, two array variables can "point to" the same actual array of values. Here's an example. The array variable named gradeAR pictured here is a char array variable. The variable gradeAR refers to a five-element array of char values. The individual elements in gradeAR are each indexed or subscripted, starting at 0 and going to 4. Now that you have a conceptual picture of what an array looks like, let's turn to the Java instructions you use to create and initialize array variables.
 **Slide 6** **Creating Arrays** Duration: 00:00:60 Advance mode: Auto	**Notes:** Creating an array is conceptually a two step process, just like creating an object (even though you can perform both steps in a single statement, just like you can with objects.) Here's what I mean. When you create an object, conceptually, you first declare an object variable, like this: Frog kermit; This variable doesn't contain an object; there is no Frog object at this point. It is simply a bucket that could store a reference to a Frog, sometime in the future. To actually create a Frog object, you need to use the new operator and the Frog constructor. Then, to "attach" the actual object to the object variable, you use the assignment operator like this: kermit = new Frog();

	Arrays work in an almost identical manner: the two steps are to declare the variable and then create the array. The syntax is slightly different though.
 **Slide 7** **Step 1: Declare the Variable** Duration: 00:01:18 Advance mode: Auto	**Notes:** Just as with objects, the first step is to declare an array variable to refer to later refer to the array elements. To declare an array variable you specify: 1. The type of elements you'll store in the array. This can be any type, object or primitive. Of course, this is exactly what you'd do to declare any kind of variable, array or not. 2. A name for the entire collection. Here you must follow the standard naming rules. There are no special names for arrays. Again, this is exactly the same as any other variable. 3. Finally, you supply a set of empty square brackets [] following either the variable name or the type name. Note that these are not parentheses. Java convention dictates placing the brackets after the type name, because it associates the brackets with the type rather than with the variable. Placing the brackets after the variable name is a holdover from the C language syntax, but even if you know C, I'd discourage using that form, since its meaning is subtly different. These brackets are what differentiates an array variable declaration from any other kind of declaration.
 **Slide 8** **Declaration Examples** Duration: 00:00:53	**Notes:** Here are a few declaration examples to give you the idea. • This example creates an array variable that holds chars. I've put the brackets after the type name. You should read this as "gradeAR is an array of char" • This example creates an array variable that holds ints. Here I've put the brackets after the variable name. Read this as "pieces is an array of int". • This example creates an array variable named deck that holds playing Cards. As you can see, the syntax is just the same whether you're creating an array of primitive types or of object references. To actually store values, however, you'll have to create the

Advance mode: Auto	actual elements that make up the array. Let's look at that now.
 **Slide 9** **Step 2: Instantiate the Array** Duration: 00:02:07 Advance mode: Auto	**Notes:** Just as with objects, the actual array is created by using the new operator. When used to create an array, though, the syntax used with new is a little different than the syntax used to create an object. When creating an object, you use an object constructor which appears on the right side of the assignment operator. An object constructor uses the name of the class, followed by parentheses and any parameters that are used by the object, such as new Frog(Color.green). Object constructors are actually methods, defined inside the class of the object you are creating. When creating an array, you use an array constructor, which is a little different than an object constructor. • First, an array constructor uses the name of the class or the name of a primitive type, followed by a set of brackets, instead of parentheses. • Inside the brackets you don't put parameters, like you do with object constructors. Instead, you put the number of elements you want your array to have. The number used to represent the array size: • can be a constant, like the 5 used for grades in this first example. The grades array has five elements, and the subscripts go from zero to four. • can be an expression or variable, like the expression nTiles * nTiles, used here for the pieces array. The pieces array has nine elements, and the subscripts go from zero to eight. • can be a variable like deckSize used to instantiate the deck array. Note though, that when you create an array that holds objects, such as deck, the array actually holds references to objects. You still have to create the object variables in a separate step. You'll learn how to do that next week. This is quite a bit of material, though, so let's do an exercise to make sure that you understand it. What I want you to be able to do at this point is create array variables of different sizes.



Slide 10

Express Yourself

Duration: 00:01:23
Advance mode: Auto

Notes:

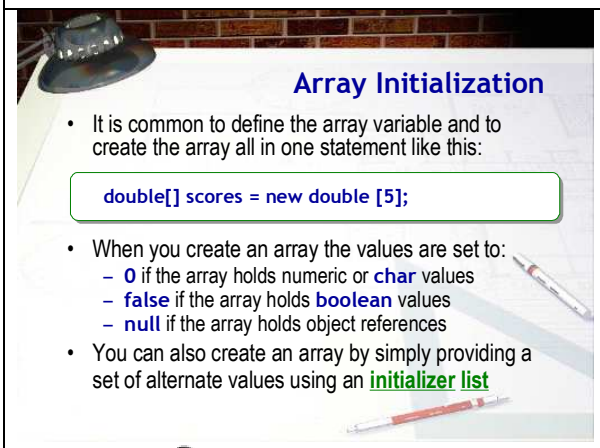
Go ahead and start your IC document for the online lecture exercises this week. Then, for Exercise 1, open Eclipse and import the project ic16 from the zip file on the class Web page. Open the file named ArrayTest and snap me a picture.

Now, inside the ArrayTest program, locate the run() method. Inside the run() method:

- declare a constant named CLASS_SIZE that represents the number of students in a class. Initialize it to 40.
- declare a one-dimensional double array variable named quizAvg that we'll use to refer to quiz score averages.
- instantiate the array using the CLASS_SIZE variable

Copy and paste your code into your IC document for Exercise 2.

Now, let's try another one using a constant. Create a one-dimensional int array named scores. Then, on the next line, instantiate the array so that it can hold 20 elements. Copy and paste your code into your IC document for Exercise 3.



Slide 11

Array Initialization

Duration: 00:01:40
Advance mode: Auto

Notes:

If you like, you can declare an array variable and create the array at the same time. This is actually quite common.

Here's what it looks like:

double[] scores = new double[5];

When you create a primitive variable, you use the assignment operator to give it an initial value. With array variables, though, we're using the assignment operator (along with new) to create the elements. How can we give the individual elements an initial value?

Well, when you first create an array using new, the values stored in each of the elements are automatically set to:

- 0 if the array holds integer or character values.
- false if the array holds boolean values.
- The value null if the array holds object references.

Note that this is different than simple variables. With a simple variable, default (automatic) initialization is only provided if the variable is a field, (declared outside of all methods). All array variables, both fields and locals, are automatically initialized when you create them.

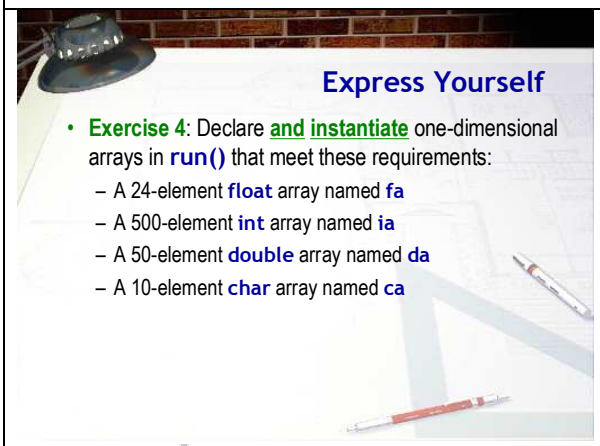
Often, however, this default initialization is not what you want. If this is the case, then you'll have to explicitly initialize your variables. You can initialize the elements in an array in two ways:

1. You can explicitly set each element by using the subscripted name of each element and storing a value in

that element. This is most often done in a loop.

2. You can provide a set of initial values in an initializer list.

Before we do that, though, let's do another exercise.



Slide 12

Express Yourself

Duration: 00:00:40

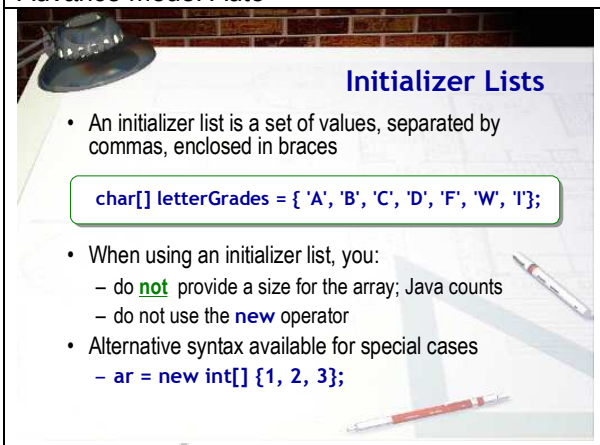
Advance mode: Auto

Notes:

Now that you know how to declare and instantiate an array variable on one line, add these variable declarations to your run() method:

- a 24-element float variable named fa
- a 500-element int array name ia
- a 50-element double array named da
- a 10-element char array named ca

When you're done, copy and paste your code into your IC document.



Slide 13

Initializer Lists

Duration: 00:01:14

Advance mode: Auto

Notes:

To explicitly provide a set of initial values, you can write each value, separated by commas, inside a set of braces; this is called an initializer list and it is an alternate way to create an array.

Here's an example declaration that shows you how to do this:

```
char[] letterGrades = {'A', 'B', 'C', 'D', 'F', 'W', 'I'};
```

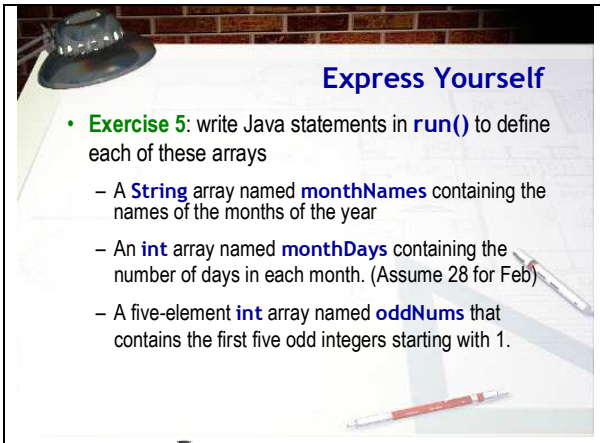
When you do this, you cannot supply a size for the array and you usually do not use the new operator.

When you write this, the Java compiler counts the number of elements in your initializer and automatically creates a seven-element array. (Make sure you don't forget the semicolon at the end of the list.)

You can also use a similar (although infrequently used) syntax, which employs both the intitializer list, and the new operator, to create something like an array literal, which can be assigned to an array variable, like this:

```
ar = new int[] { 1, 2, 3 };
```

This creates an anonymous array holding the values 1, 2, and 3, and assigns a reference to the variable ar. Note that you must use this syntax if you want to assign a series of values to an existing array variable.



Slide 14

Express Yourself

Duration: 00:01:09
Advance mode: Auto

Notes:

OK, let's finish up this mini-lecture with a final exercise. For exercise 5, write Java statements in the run() method to define each of these arrays.

- Start with a String array containing the names of the months, "January", "February" and so on. Name it monthNames.
- Next, create an int array named monthDays that contains the number of days in each month. Assume 28 for February.
- Finally, create a five-element int array named oddNums. Initialize oddNums so that it contains the first five odd integers, starting, naturally, with 1.

When you're done, make sure your code is syntactically correct by saving it and checking the Eclipse doesn't add any red squigglies. Then, copy your code and paste it into your IC document for Exercise 5.

Now that we have some arrays, now let's see how to access the data they contain.

